



Laboratório 5: autorização com OAuth2

25/11/2025

Nesse laboratório será demonstrado como criar uma aplicação Java para consumir a API REST do GitHub¹ e listar os repositórios de um usuário autenticado em uma página HTML. Para isso, será utilizado o protocolo OAuth2 para autenticação e autorização de usuários.

1 Criando um aplicativo OAuth na sua conta do GitHub

Para configurar o OAuth2, é necessário criar um aplicativo OAuth no GitHub e obter as credenciais de acesso. Abaixo estão os passos resumidos para criar um aplicativo²:

1. Acesse as configurações da sua conta no GitHub;
2. Clique em *Developer settings* no menu lateral esquerdo e em seguida clique em *OAuth Apps*;
3. Clique no botão *Register a new application*, caso você ainda não tenha criado um aplicativo ou em *New OAuth App* caso você já tenha criado um aplicativo anteriormente;
4. Em *Application name* informe o nome do aplicativo. Por exemplo, *Meu aplicativo Java*;
5. Em *Homepage URL* informe a URL do site do aplicativo. No nosso caso será um aplicativo local e a URL será `http://localhost:8080`;
6. Em *Authorization callback URL* informe a URL de redirecionamento após a autenticação do usuário. No nosso exemplo será `http://localhost:8080/login/oauth2/code/github`;
7. Clique no botão *Register application* para criar o aplicativo;
8. Anote o *Client ID* e o *Client Secret* gerados pelo GitHub, pois serão necessários para configurar o aplicativo Java que consumirá a API REST do GitHub.

2 Criando um aplicativo Java para consumir a API REST do GitHub

Para consumir a API REST do GitHub, vamos criar um aplicativo Java utilizando o *framework Spring Boot* e a biblioteca *Spring Security*. Acesse o endereço <https://start.spring.io/> e crie um projeto com as seguintes dependências:

- Spring Web
- Spring Security
- Thymeleaf
- OAuth2 Client
- Spring Reactive Web

¹<https://docs.github.com/pt/rest>

²No endereço <https://docs.github.com/pt/apps/oauth-apps/building-oauth-apps/creating-an-oauth-app> você encontrará um tutorial detalhado de como criar um aplicativo no GitHub.

A configuração do Spring Boot pode ser feita no arquivo `application.properties` que está dentro do diretório `src/main/resources` do projeto. Nesse arquivo, devemos informar o *Client ID* e o *Client Secret* gerados por você na Seção 1, além de outras configurações necessárias para o funcionamento do aplicativo. Edite o arquivo `application.properties` conforme o exemplo da Listagem 1.

Listagem 1: Arquivo de configuração do Spring Boot

```
spring.application.name=demo
spring.security.oauth2.client.registration.github.client-name=Laboratorio de Autorizacao
spring.security.oauth2.client.registration.github.client-id=seu-client-id
spring.security.oauth2.client.registration.github.client-secret=seu-client-secret
spring.security.oauth2.client.registration.github.scope=user,repo
```

Na sequência vamos criar os dois arquivos HTML que serão utilizados para autenticação e listagem dos repositórios do usuário. O arquivo `index.html`, apresentado na Listagem 2, é a página inicial do aplicativo e contém um *link* para autenticação do usuário, caso não esteja autenticado, ou um *link* para listar os repositórios do usuário autenticado.

Listagem 2: Página `index.html`

```
<!DOCTYPE html>
<html lang="pt_BR" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laboratório sobre Autorização</title>
  <style>
    .container {
      max-width: 400px;
      margin: 60px auto;
      background: #fff;
      border-radius: 8px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      padding: 32px 24px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Laboratório sobre autorização</h1>

    <div th:if="${nome == null}">
      <p>Você não está autenticado.</p>
      <a href="/oauth2/authorization/github">Fazer login com GitHub</a>
    </div>
    <div th:if="${nome != null}">
      <p>Olá, <strong th:text="${nome}">Usuário</strong>! Você fez login com sucesso pelo
      GitHub.</p>
      <p>Veja seus <a href="/repos">repositórios clicando aqui</a>.</p>
      <a href="/logout" >Sair</a>
    </div>
  </div>
</body>
</html>
```

O arquivo `repositorios.html`, apresentado na Listagem 3, é a página que lista os repositórios GitHub do usuário autenticado.

Listagem 3: Página repositorios.html

```
<!DOCTYPE html>
<html lang="pt_BR" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meus Repositórios do GitHub</title>
  <!-- Bootstrap CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="
    stylesheet">
</head>
<body class="bg-light">
  <div class="container py-5">
    <h1 class="mb-4 text-center">Meus Repositórios do GitHub</h1>

    <div th:if="${#lists.isEmpty(repos)}" class="alert alert-warning text-center">
      Você não possui repositórios ou ocorreu um erro ao buscar seus repositórios.
    </div>

    <div th:unless="${#lists.isEmpty(repos)}">
      <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
        <div class="col" th:each="repo : ${repos}">
          <div class="card h-100 shadow-sm">
            <div class="card-body">
              <h5 class="card-title">
                <a th:href="${repo.html_url}" th:text="${repo.name}" target="
                  _blank" class="fw-bold text-decoration-none"></a>
              </h5>
              <p class="card-text" th:text="${repo.description != null ? repo.
                description : 'Sem descrição.'}"></p>
            </div>
            <div class="card-footer bg-transparent border-0 text-end">
              <a th:href="${repo.html_url}" target="_blank" class="btn btn-outline-
                primary btn-sm">Ver no GitHub</a>
            </div>
          </div>
        </div>
      </div>
      <div class="mt-4 text-center">
        <a href="/" class="btn btn-secondary">Voltar para a página inicial</a>
      </div>
    </div>
  </body>
</html>
```

Na sequência vamos criar a classe *SecurityConfig.java* (veja Listagem 4) que contém a configuração do Spring Security. Na classe também ficará com o mapeamento da URL de redirecionamento após a autenticação do usuário, no caso a página *index.html*.

Listagem 4: Configuração do Spring Security

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.security.web.SecurityFilterChain;
```

```

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

/**
 * Classe de configuração do Spring Security
 */
@Controller
public class SecurityConfig {

    /**
     * Configuração do filtro de segurança para habilitar o login OAuth2
     */
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        // Autoriza qualquer requisição autenticada
        http.authorizeHttpRequests(
            authorizeRequests -> authorizeRequests.anyRequest().authenticated()
        ).oauth2Login(Customizer.withDefaults()); // Habilita o login OAuth2
        return http.build();
    }

    /**
     * Método que retorna a página inicial da aplicação com o nome do usuário autenticado.
     *
     * @param model Objeto Model para adicionar atributos à página HTML
     * @param principal Objeto OAuth2User com os dados do usuário autenticado
     * @return String com o nome da página HTML a ser exibida na aplicação web
     */
    @GetMapping("/")
    public String index(Model model, @AuthenticationPrincipal OAuth2User principal) {
        if (principal != null) {
            model.addAttribute("nome", principal.getAttribute("name"));
        }
        return "index";
    }
}

```

Crie a classe *WebClientConfig.java* (veja Listagem 5) para colocar a configuração do *WebClient*, objeto usado para fazer chamadas à API REST do GitHub.

Listagem 5: Configuração de WebClient para fazer chamadas à API do GitHub

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;

/**
 * Classe de configuração do WebClient para realizar chamadas à API do GitHub.
 */
@Configuration
public class WebClientConfig {

    /**
     * Método que retorna um objeto WebClient.Builder para realizar chamadas à API do GitHub.
     */
    @Bean
    public WebClient.Builder webClientBuilder(){
        return WebClient.builder();
    }
}
```

Crie a classe *GitHubApiController.java* (veja Listagem 6) para fazer as chamadas à API REST do GitHub. Nessa classe, o método *getRepos()* faz uma chamada à API do GitHub para listar os repositórios do usuário autenticado, os quais são exibidos na página *repositorios.html*.

Listagem 6: Classe de controle para chamadas à API do GitHub

```
package com.example.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.security.oauth2.client.OAuth2AuthorizedClient;
import org.springframework.security.oauth2.client.annotation.RegisteredOAuth2AuthorizedClient;
import org.springframework.security.oauth2.core.OAuth2AccessToken;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.reactive.function.client.WebClient;

import java.util.List;

/**
 * Classe de controle para realizar chamadas à API do GitHub.
 */
@Controller
public class GitHubApiController {

    // Injeção de dependência do WebClient.Builder
    @Autowired
    private WebClient.Builder webClientBuilder;

    /**
     * Método que retorna a página 'repositorios.html' com a lista de repositórios do usuário autenticado.
     * @param model Objeto Model para adicionar atributos à página HTML
     */
}
```

```

    * @param authorizedClient Objeto OAuth2AuthorizedClient com as informações do cliente
    autorizado
    * @return String com o nome da página HTML a ser exibida na aplicação web
    */
@GetMapping("/repos")
public String getRepos(Model model, @RegisteredOAuth2AuthorizedClient("github")
    OAuth2AuthorizedClient authorizedClient) {
    OAuth2AccessToken accessToken = authorizedClient.getAccessToken();
    // Chamada à API do GitHub para obter os repositórios
    List<Object> repos = webClientBuilder.build()
        .get()
        .uri("https://api.github.com/user/repos")
        .headers(headers -> headers.setBearerAuth(accessToken.getTokenValue()))
        .retrieve()
        .bodyToMono(new ParameterizedTypeReference<List<Object>>() {})
        .block();

    // Adiciona a lista de repositórios ao modelo
    model.addAttribute("repos", repos);
    return "repositorios"; // Retorna a página 'repositorios.html'
}
}

```

Pronto! Agora pode executar a aplicação pelo terminal com o comando `./gradlew bootRun` ou pela IDE.

- Acesse a URL <http://localhost:8080> no navegador e clique no *link* para autenticar o usuário.